



American International University-Bangladesh (AIUB)

Department of Computer Science

Faculty of Science & Technology (FST)

Summer 2026

Section: F

Software Quality Assurance and Testing

**FRESHCART: E-COMMERCE & GROCERY DELIVERY
MANAGEMENT SYSTEM**

A Report Submitted By

SN	Student Name	Student ID
1	MD. AL AMIN	23-52210-2
2	ZANNATUN NUR	23-50183-1
3	MD.REDUONE AHMED	22-49176-3
4	SANMUN NAHAR JOLY	23-51275-1

Under the Supervision of

Md. Anwarul Kabir

Software Test Plan

for

FreshCart: E-Commerce & Grocery Delivery Management System

Version 1.0 approved

Prepared by MD. AL AMIN, ZANNATUN NUR, MD.REDUONE AHMED,
SANMUN NAHAR JOLY

American International University-Bangladesh (AIUB)

September 14, 2026

Checked By Industry Personnel

Name:

Designation:

Company:

Sign:

Date:

Table of Contents

Revision History	3
1. TEST PLAN IDENTIFIER: RS-MTP01.3	4
2. REFERENCES	4
3. INTRODUCTION	4
Background to the Problem.....	4
Solution to the Problem.....	4
4. REQUEIREMNT SPECIFICATION	5
4.1 System Features	5
4.2 System Quality Attributes.....	7
4.3 System Interface.....	7
4.4 Project Requirements	9
5. FEATURES NOT TO BE TESTED.....	10
6. TESTING APPROACH.....	11
6.1 Testing Levels.....	11
6.2 Test Tools.....	11
6.3 Meetings.....	11
7. TEST CASES/TEST ITEMS.....	12
8. ITEM PASS/FAIL CRITERIA.....	19
9. TEST DELIVERABLES	19
10. STAFFING AND TRAINING NEEDS.....	19
11. RESPONSIBILITIES	19
12. TESTING SCHEDULE	20
13. PLANNING RISKS AND CONTINGENCIES	21
14. APROVALS	21

Revision History

Revision	Date	Updated by	Update Comments
0.1	September 1, 2026	ZANNATUN NUR	First Draft
0.2	September 2, 2026	SANMUN NAHAR JOLY	Second Draft
0.3	September 3, 2026	MD. AL AMIN	Third Draft
0.4	September 4, 2026	MD.REDUONE AHMED	Fourth Draft
0.5	September 5, 2026	ZANNATUN NUR	Fifth Draft

1.0	September 8, 2026	MD. AL AMIN	Initial version of the Software Test Plan for FreshCart (E-Commerce & Grocery Delivery Management System) approved
-----	-------------------	-------------	--

1. TEST PLAN IDENTIFIER:RS-MTP01.3

2. REFERENCES

- FreshCart Project and Setup Guide, submitted with the project source code
Link: [GitHub - zinpacchi-10/SQAT-Projects · GitHub](#)

3. INTRODUCTION

Background to the Problem

- Small and medium grocery shops in Bangladesh sell mostly face-to-face, by phone, or through informal Facebook pages. Customers can't browse a real catalog, compare prices across sellers, track orders, or pay online. Shop owners have no dashboard for stock, orders, or delivery. Result: slow, error-prone deals on both sides.
- Root cause: no shared, role-aware platform linking customers, sellers, delivery staff, and admin in one system. Without it, sellers can't reach new customers well, delivery stays uncoordinated and untracked, and customers have no way to check order status, request returns, or resolve disputes. This hurts customer trust and seller revenue.

Solution to the Problem

- We propose FreshCart, a multi-role e-commerce and grocery delivery web application with four dedicated roles: Customer, Seller, Delivery Manager and Admin, each with a role-specific dashboard.
This solution is appropriate because it gives every stakeholder the exact tools they need catalog browsing and checkout for customers, inventory and order management for sellers, dispatch and tracking for delivery managers, and platform oversight for admins inside one connected system. It is feasible because it is built with widely available, license-free technology (PHP, MySQL, HTML/CSS/JavaScript) that runs on ordinary XAMPP hosting, so it meets the business objective of a low-cost, quickly deployable local marketplace.
- FreshCart is a hand-rolled MVC web application (no external framework) that lets customers browse and buy grocery products online, lets sellers manage their own shop, products and orders, lets a delivery team dispatch and track deliveries, and lets an admin moderate the whole

marketplace. Its goals are to digitize the ordering process, keep stock accurate in real time, and give every role visibility into the status of an order from placement to delivery.

- Established platforms such as Chaldal, Daraz and Shwapno online solve parts of this problem at a national scale, but they are closed, proprietary systems that a small project or a local shop cannot adopt, extend or self-host.

FreshCart is presented as a self-contained, open, self-hostable alternative built specifically to demonstrate and test the same core workflow: catalog, cart, checkout, fulfilment and delivery, under a single MySQL/PHP stack.

4. REQUIREMENT SPECIFICATION

4.1 System Features

- The following functional requirements were implemented in FreshCart and are covered by this test plan:
- 1. Customer Account Registration & Login
Functional Requirements
 - 1.1 The system shall allow a visitor to register as a Customer with name, email, phone number and password, and to log in with email and password.
 - 1.2 Passwords shall be hashed with PHP's `password_hash()` and verified with `password_verify()`; the system shall never store or display plaintext passwords.
 - 1.3 If the account is deactivated (`is_active = 0`), or belongs to a seller whose approval status is pending, rejected or suspended, the system shall block login and show the specific reason.

Priority Level: High

Precondition: User has a valid account; for sellers, the account must be approved by the Admin

- 2. Product Catalog Browsing, Search & Cart
Functional Requirements
 - 1.4 The system shall allow a customer to browse, search, filter and sort available products by category, price and seller.
 - 1.5 The system shall allow a customer to add a product to the cart and to the wishlist using AJAX, without a full page reload.
 - 1.6 The system shall display product details, images and existing customer reviews on the product detail page.

Priority Level: High

Precondition: Product catalog contains at least one active, available product

- 3. Checkout, Coupon Validation & Order Placement
Functional Requirements

- 1.7 The system shall validate a coupon code entered at checkout via AJAX and apply the discount to the order total before the order is submitted.
- 1.8 The system shall allow the customer to place an order using Cash on Delivery or Card, and shall re-check stock for every item inside a database transaction before decrementing stock and confirming the order.
- 1.9 The system shall provide live AJAX polling of order status on the order detail page, and shall allow the customer to cancel an order while it is still pending.

Priority Level: High

Precondition: Cart contains at least one in-stock item and a valid delivery address

- 4. Seller Product & Order Management

- Functional Requirements

- 1.10 The system shall allow an approved seller to create, edit, and remove their own products, including a primary image and extra images.
 - 1.11 The system shall allow a seller to update product stock quantity and toggle product availability instantly via AJAX.
 - 1.12 The system shall allow a seller to view incoming orders and move each order forward from confirmed to shipped with a tracking note, and to approve or reject return requests (an approval automatically restocks the item).

Priority Level: High

Precondition: Seller account is approved by Admin and has at least one product listed

- 5. Delivery Dispatch & Status Tracking

- Functional Requirements

- 1.13 The system shall allow a Delivery Manager to assign a delivery agent to a placed order from the dispatch queue using AJAX.
 - 1.14 The system shall prevent assignment of an inactive delivery agent to any order.
 - 1.15 The system shall allow the assigned agent's status to be updated through picked up, in transit, delivered, or failed, with failed deliveries available for reassignment.

Priority Level: Medium

Precondition: Order has passed checkout and a delivery zone with an active agent exists

- 6. Admin Seller Approval & Platform Moderation

- Functional Requirements

- 1.16 The system shall allow the Admin to approve, reject or suspend a seller registration, and to set a per-seller commission override or use the platform default commission.

- 1.17 The system shall allow the Admin to manage categories, moderate (force-remove) product listings, curate featured products, and manage customer and delivery-manager accounts.
- 1.18 The system shall allow the Admin to resolve customer disputes and publish platform-wide coupons and announcements.

Priority Level: Medium

Precondition: Admin is logged in with an active administrator account

4.2 System Quality Attributes

- The system was built and tested against the following quality attributes:
- Security: *Every SQL query in the application shall go through mysqli prepared statements (no string-concatenated SQL), passwords shall be hashed, and every controller action shall enforce role-based access control server-side via Controller::requireRole().*
- Reliability: *Stock quantity shall never go negative: it is enforced by a database CHECK constraint and re-validated inside a database transaction in OrderModel::placeOrder() before any stock is decremented.*
- Usability: *A returning customer shall be able to search for a product, add it to the cart, and reach the checkout page in no more than four clicks from the homepage.*
- Performance: *AJAX-driven actions (add to cart, wishlist toggle, coupon validation, stock update, delivery status update) shall update the page without a full reload and shall complete within 2 seconds on a local XAMPP environment.*

4.3 System Interface

- FreshCart exposes four separate, role-based web interfaces reached after login: the Customer storefront (home, product catalog, product detail, cart, checkout, order tracking, wishlist, profile), the Seller dashboard (product CRUD, stock & coupons, order fulfilment, analytics), the Delivery Manager console (dispatch queue, active deliveries board, agents & zones), and the Admin console (platform KPIs, seller approval, category & coupon management, disputes, reports). Each view is rendered server-side by a PHP controller/view pair and enhanced with AJAX for the interactive actions listed under System Features. Annotated screenshots of each dashboard are attached as a separate appendix.

Login Page

Shop Log In [Become a Seller](#)

Log In

Welcome back to FreshCart Marketplace.

Email

Password

[Log In](#)

No account? [Sign up as customer](#) or [register your shop](#).

Demo accounts (password: Password123)

Admin: admin@grocery.test

Delivery Manager: delivery@grocery.test

Seller: seller@grocery.test

Customer: customer@grocery.test

© 2026 FreshCart Marketplace — E-Commerce & Grocery Delivery Management System (Academic Project)

Registration Page

Shop Log In [Become a Seller](#)

Create Your Account

Shop groceries from multiple local sellers.

Full Name

Email

Phone

Password

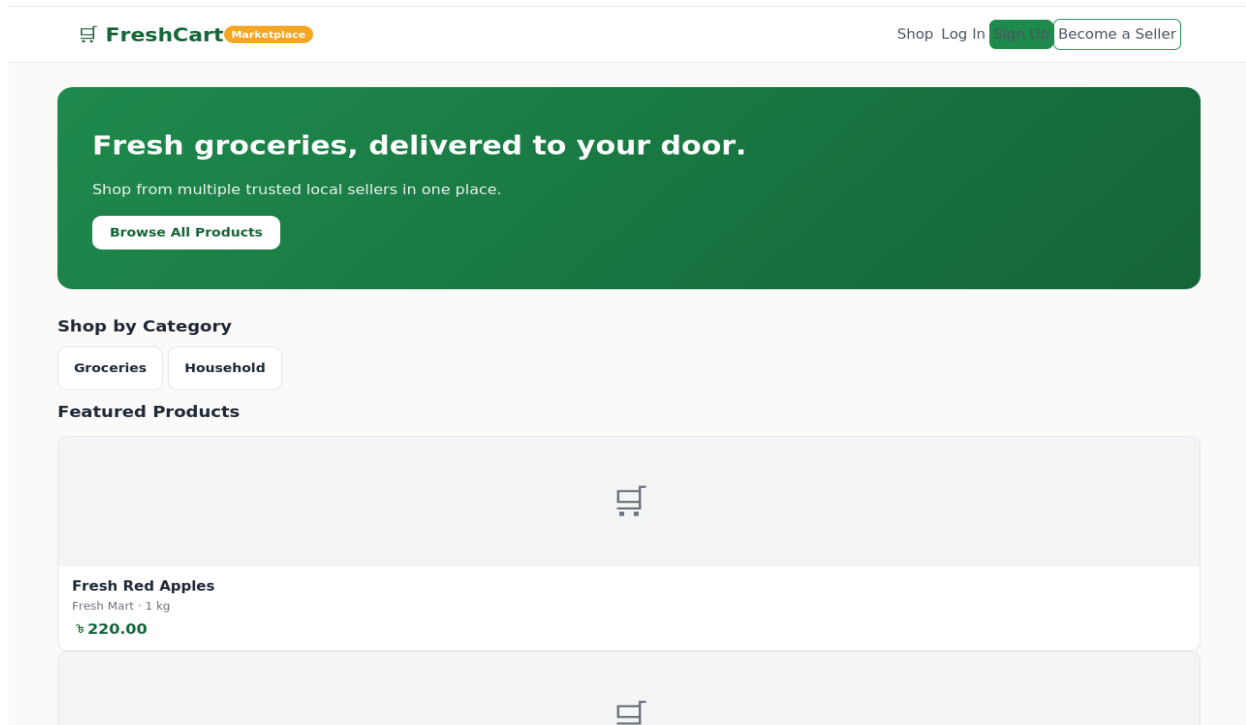
Minimum 6 characters.

[Create Account](#)

Already have an account? [Log in](#)

© 2026 FreshCart Marketplace — E-Commerce & Grocery Delivery Management System (Academic Project)

Home/Landing Page



4.4 Project Requirements

Requirement	Description
Budget	The project shall be developed within an estimated budget of BDT 1,50,000 covering development, hosting, testing, and other necessary resources; no third-party AI service cost applies.
Development Time	The total development period shall be approximately 12 weeks, including requirements analysis, design, development, database setup, testing, and deployment.
Development Team	The project shall require a 4-member student team sharing developer and tester roles across the Customer, Seller, Delivery Manager, and Admin modules.
Development Environment	The application shall be developed and tested using XAMPP (Apache, PHP, MySQL/MariaDB) with phpMyAdmin for database management on standard local development machines.
Hardware Resources	Development and testing shall require standard computers capable of running the XAMPP stack,

	a modern web browser, and Git for version control.
Software Resources	The project shall use PHP, MySQL/MariaDB, HTML/CSS/vanilla JavaScript, and Git/GitHub for version control, with no external framework dependency.
Database	A MySQL/MariaDB database (grocery_delivery_system) shall be required to store users, products, orders, deliveries, coupons, reviews, and other application data.
Internet Connectivity	Internet connectivity shall be required for Git/GitHub version control and browser-based testing; the core application itself runs on a local XAMPP server.
Security	User credentials and personal information shall be protected through password hashing, mysqli prepared statements, and server-side role-based access control.
Testing Resources	Sufficient time and resources shall be allocated for unit, system/integration, and acceptance testing across all four roles before final submission.
Deployment Environment	The completed application shall be deployed under a XAMPP htdocs environment capable of supporting the application's database, session-based authentication, and media-upload requirements.

5. FEATURES NOT TO BE TESTED

The following is a list of the areas that will not be specifically addressed. All testing in these areas will be indirect as a result of other testing efforts. For example:

- Real payment gateway integration. Checkout supports Cash on Delivery and Card as payment methods, but no live third-party payment gateway (e.g. bKash, SSLCommerz, Stripe) is connected; card details are not actually charged.
- Third-party email/SMS delivery. Order confirmation and notification content is generated and stored in-app (notifications table) but is not routed through an external email or SMS provider.
- Web server and OS-level configuration (Apache, PHP, MySQL/MariaDB versions, XAMPP installation). These are environment prerequisites documented in the README and are outside the scope of functional/system testing.

6. TESTING APPROACH

6.1 Testing Levels

- Testing for the FreshCart project consists of Unit, System/Integration (combined) and Acceptance test levels. As this is a 4-member student group project with no dedicated budget for an independent tester, most testing is carried out jointly by the group members, with each member primarily responsible for testing the role(s)/modules they implemented and at least one other member cross-checking each module.
- UNIT Testing is performed by the group member who built the corresponding controller/model/view (e.g. AuthController, OrderModel), and reviewed by the group's designated test coordinator. Evidence of unit testing (test case list, screenshots, and any defects found) is shared with the group before the module is merged into the shared codebase.
- SYSTEM/INTEGRATION Testing is performed by the whole group against the live application running on a shared XAMPP/MySQL setup, verifying end-to-end flows that cross roles (e.g. customer places an order → seller ships it → delivery manager assigns and updates it). A module enters System/Integration testing only after all critical defects found during unit testing have been corrected. A module may still have up to two Major defects as long as they do not block further testing (i.e. a workaround exists).
- ACCEPTANCE Testing is performed by the course instructor/industry personnel reviewer and, where possible, informal end users (classmates acting as customer/seller/delivery agent), with the group demonstrating each of the four role-based dashboards.

6.2 Test Tools

Google Chrome – manual functional and UI testing across roles; phpMyAdmin – verifying database state (stock, orders, users) before/after each test; Browser DevTools (Network tab) – inspecting AJAX request/response payloads for add-to-cart, coupon validation, stock updates and delivery status changes; XAMPP Control Panel – managing the Apache/MySQL test environment; Git/GitHub – version control of source code and test documentation.

6.3 Meetings

The 4-member group will meet at least once a week (in person or online) to review testing progress, discuss defects found, and re-assign modules if a member falls behind. The group will meet with the course instructor/supervisor according to the course's scheduled consultation slots. Additional ad-hoc meetings will be called if a blocking defect is found close to a submission deadline.

7. TEST CASES/TEST ITEMS

1. TC_01

Project Name: FreshCart		Test Designed by: MD. AL AMIN		
Test Case ID: TC_01		Test Designed date: September 3, 2026		
Test Priority (Low, Medium, High): High		Test Executed by: MD. AL AMIN		
Module Name: Login Session		Test Execution date: September 6, 2026		
Test Title: Verify customer login with valid email and password				
Description: Test the FreshCart login page (AuthController::login) for the customer role				
Precondition (If any): Customer has an existing active account (email: customer@grocery.test)				
Test Steps	Test Data	Expected Results	Actual Results	Status (Pass/Fail)
1. Navigate to the FreshCart login page (index.php?url=auth/login) 2. Enter registered email address 3. Enter correct password 4. Click the 'Login' button	Email: customer@grocery.test / Password: Password123	Customer is authenticated and redirected to the customer dashboard (customer/dashboard); a PHP session is created	As expected .	Pass
Post Condition: A session is created for the user; role-based redirect sends the customer to their dashboard; session persists until logout.				

2. TC_02

Project Name: FreshCart	Test Designed by: MD. AL AMIN
Test Case ID: TC_02	Test Designed date: September 3, 2026
Test Priority (Low, Medium, High): High	Test Executed by: MD. AL AMIN

Module Name: Checkout & Coupon Validation		Test Execution date: September 6, 2026		
Test Title: Verify checkout applies a valid coupon and correctly decrements stock				
Description: Test the customer checkout flow (CustomerController::checkout, OrderModel::placeOrder) with a valid coupon code				
Precondition (If any): Customer is logged in, cart contains 1 unit of a product with stock_qty >= 1, and an active, unexpired coupon exists				
Test Steps	Test Data	Expected Results	Actual Results	Status (Pass/Fail)
5. Open Cart and click 'Proceed to Checkout' 6. Enter the coupon code and click 'Apply' (AJAX) 7. Select payment method 'Cash on Delivery' and a delivery address 8. Click 'Place Order'	Coupon code: SAVE10 (10% off); product stock before order: 5 units	Discount is applied to the order total via AJAX without a page reload; the order is created; product stock_qty decreases by the ordered quantity (5 → 4); the new order appears under the customer's Orders list with status 'Pending'	As expected.	Pass
Post Condition: An order_items row links the order to the product; products.stock_qty is updated inside a database transaction; coupon usage is recorded.				

3. TC_03

Project Name: FreshCart	Test Designed by: ZANNATUN NUR
Test Case ID: TC_03	Test Designed date: September 3, 2026
Test Priority (Low, Medium, High): Medium	Test Executed by: ZANNATUN NUR
Module Name: Seller Inventory Management	Test Execution date: September 6, 2026
Test Title: Verify a seller can update stock and toggle availability via AJAX	

Description: Test the seller product list AJAX stock-update and availability-toggle endpoints (SellerController)				
Precondition (If any): Seller is logged in with an approved account and owns at least one product				
Test Steps	Test Data	Expected Results	Actual Results	Status (Pass/Fail)
9. Navigate to Seller > Products 10. Edit the stock quantity field for a product and confirm 11. Observe the update applied without a page reload 12. Click the product's 'Available' toggle to switch it off	New stock quantity: 25 (previous value: 10)	Stock quantity updates to 25 immediately via AJAX; the product is marked unavailable and no longer appears in the customer-facing catalog	As expected.	Pass
Post Condition: products.stock_qty and products.is_available are updated in the database; the change is reflected instantly in both the seller product list and the customer catalog.				

4. TC_04

Project Name: FreshCart			Test Designed by: MD.REDUONE AHMED	
Test Case ID: TC_04			Test Designed date: September 3, 2026	
Test Priority (Low, Medium, High): Medium			Test Executed by: MD.REDUONE AHMED	
Module Name: Delivery Dispatch & Assignment			Test Execution date: September 6, 2026	
Test Title: Verify an inactive delivery agent cannot be assigned to an order				
Description: Test the dispatch queue AJAX agent-assignment endpoint (DeliveryController)				
Precondition (If any): Delivery Manager is logged in; a pending order exists in the dispatch queue; at least one delivery agent has status 'inactive'				
Test Steps	Test Data	Expected Results	Actual Results	Status (Pass/Fail)

13. Navigate to Delivery > Dispatch Queue 14. Select a pending order from the queue 15. Attempt to assign the inactive delivery agent via the AJAX assignment control 16. Observe the system response	Delivery agent status: inactive	The system rejects the assignment and shows an error message; the order remains unassigned in the dispatch queue	As expected.	Pass
Post Condition: No row is created in delivery_assignments for the inactive agent; the order status remains unchanged.				

5. TC_05

Project Name: FreshCart		Test Designed by: ZANNATUN NUR		
Test Case ID: TC_05		Test Designed date: September 4, 2026		
Test Priority (Low, Medium, High): High		Test Executed by: ZANNATUN NUR		
Module Name: Seller Order Fulfilment		Test Execution date: September 7, 2026		
Test Title: Verify a seller can confirm and ship a pending order with a tracking note				
Description: Test the seller order-fulfilment flow (SellerController::updateOrderStatus) moving an order from Confirmed to Shipped				
Precondition (If any): Seller is logged in with an approved account; an order containing the seller's product exists with status 'Pending'				
Test Steps	Test Data	Expected Results	Actual Results	Status (Pass/Fail)

Navigate to Seller > Orders Open the pending order and click 'Confirm' Click 'Ship' and enter a tracking note Observe the order status change	Tracking note: 'Dispatched via Sundarban Courier, expected 2 days'	Order status updates from Pending to Confirmed to Shipped without a page reload; the tracking note is saved and visible to the customer on the order detail page	As expected.	Pass
Post Condition: orders.status is updated in the database at each transition; the tracking note is stored and shown on the customer's order tracking page				

6. TC_06

Project Name: FreshCart			Test Designed by: MD.REDUONE AHMED	
Test Case ID: TC_06			Test Designed date: September 4, 2026	
Test Priority (Low, Medium, High): Medium			Test Executed by: MD.REDUONE AHMED	
Module Name: Delivery Status Tracking			Test Execution date: September 7, 2026	
Test Title: Verify a delivery agent's status can be updated through picked up, in transit and delivered				
Description: Test the active-deliveries board AJAX status-update endpoint (DeliveryController)				
Precondition (If any): Delivery Manager is logged in; an order has already been assigned to an active delivery agent				
Test Steps	Test Data	Expected Results	Actual Results	Status (Pass/Fail)

Navigate to Delivery > Active Deliveries Select the assigned order Update status to 'Picked Up' via AJAX Update status to 'In Transit', then to 'Delivered'	Delivery agent: active; Order ID: #1042	Each status transition updates instantly without a page reload; once 'Delivered' is set, the order leaves the active- deliveries board and appears in Delivery History with a delivered timestamp	As expected.	Pass
Post Condition: delivery_assignments.status is updated in the database at each step; the order's final status and delivered timestamp appear correctly in Delivery History				

7. TC_07

Project Name: FreshCart		Test Designed by: SANMUN NAHAR JOLY		
Test Case ID: TC_07		Test Designed date: September 5, 2026		
Test Priority (Low, Medium, High): High		Test Executed by: SANMUN NAHAR JOLY		
Module Name: Admin Seller Approval		Test Execution date: September 8, 2026		
Test Title: Verify the admin can approve a pending seller registration				
Description: Test the admin seller-approval workflow (AdminController::approveSeller)				
Precondition (If any): Admin is logged in with an active administrator account; a seller account exists with approval status 'pending'				
Test Steps	Test Data	Expected Results	Actual Results	Status (Pass/Fail)

Navigate to Admin > Sellers Open the pending seller's detail page Click 'Approve' Confirm the action	Seller shop name: 'Green Valley Grocers'; current status: pending	The seller's status changes to 'approved'; the seller can now log in and manage products; a notification is created for the seller	As expected.	Pass
Post Condition: sellers.is_approved is updated to 'approved' in the database; the seller's login is no longer blocked				

8. TC_08

Project Name: FreshCart			Test Designed by: SANMUN NAHAR JOLY	
Test Case ID: TC_08			Test Designed date: September 5, 2026	
Test Priority (Low, Medium, High): Medium			Test Executed by: SANMUN NAHAR JOLY	
Module Name: Admin Platform Moderation			Test Execution date: September 8, 2026	
Test Title: Verify the admin can create a platform-wide coupon and a new product category				
Description: Test the admin coupon-creation and category-management screens (AdminController)				
Precondition (If any): Admin is logged in with an active administrator account				
Test Steps	Test Data	Expected Results	Actual Results	Status (Pass/Fail)
Navigate to Admin > Coupons and create a new coupon Navigate to Admin > Categories and add a new	Coupon code: 'WELCOME15', discount 15%, valid until Dec 31, 2026; Category name:	The coupon is created with seller_id NULL and created_by_admin = 1 and is applicable at	As expected.	Pass

category Save both changes	'Organic Produce'	checkout across all sellers; the new category appears in the customer catalog filter		
Post Condition: A new row is created in coupons (platform-wide) and in categories; both are immediately visible on the customer storefront				

ITEM PASS/FAIL CRITERIA

A test item (feature/module) is considered PASS when all of its designed test cases execute with the “Expected Results” matching the “Actual Results” and no Critical or High severity defect remains open against it. Testing for the whole project is considered complete when every functional requirement listed in Section 4.1 has at least one executed test case, at least 90% of all test cases pass, and no Critical defect is open in the Login/Authentication, Checkout/Order-placement, or Stock-decrement modules.

8. TEST DELIVERABLES

- Acceptance test plan
- System/Integration test plan
- Unit test plans/turnover documentation
- Screen prototypes
- Report mock-ups
- Defect/Incident reports and summaries
- Test logs and turnover reports

9. STAFFING AND TRAINING NEEDS

As a 4-member student group, there is no separate dedicated tester; all four members share unit, system/integration and acceptance testing duties alongside development, in addition to one member acting as test coordinator who consolidates the test case list and defect log for submission. In order to provide complete and proper testing the following areas need to be addressed in terms of training.

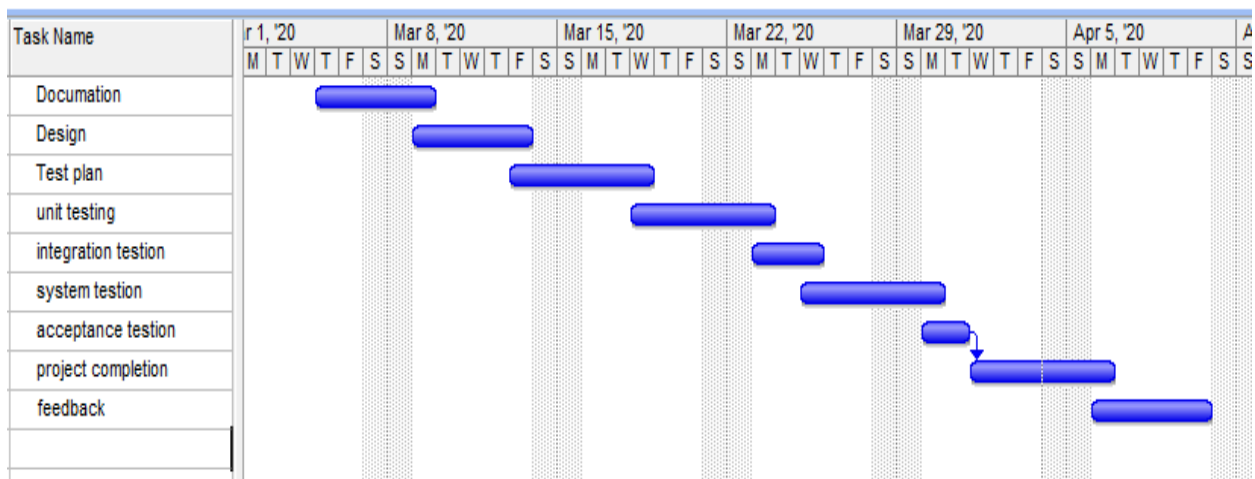
- All group members need a working knowledge of the XAMPP stack (Apache, MySQL/MariaDB, phpMyAdmin), how to import database/schema.sql, and how to read AJAX requests in browser DevTools, since most interactive features in this project are AJAX-driven.
- Each member must be briefed on the demo accounts and workflow for the role(s) they did not build (Customer, Seller, Delivery Manager, Admin) so that cross-role, end-to-end test cases can be executed by anyone in the group.

10. RESPONSIBILITIES

	TM	PM	Dev Team	Test Team	Client
Acceptance test Documentation & Execution	X	X		X	X
System/Integration test Documentation & Exec.	X		X	X	
Unit test documentation & execution	X		X	X	
System Design Reviews	X	X	X	X	X
Detail Design Reviews	X	X	X	X	
Test procedures and rules	X	X	X	X	
Screen & Report prototype reviews			X	X	X
Change Control and regression testing	X	X	X	X	X

11. TESTING SCHEDULE

Time has been allocated within the project plan for the following testing activities. The specific dates and times for each activity are defined in the project plan timeline. The persons required for each process are detailed in the project timeline and plan as well. Coordination of the personnel required for each task, test team, development team, management and customer will be handled by the project manager in conjunction with the development and test team leaders. Schedule must be done using any PM tool.



12. PLANNING RISKS AND CONTINGENCIES

- Limited group availability. All four group members also have overlapping exam schedules and other course deadlines during the semester, which may delay some rounds of testing. Should this happen, the group will prioritize re-testing Critical/High severity modules (Login, Checkout, Stock management) first and will not skip acceptance testing to meet a deadline; the schedule will be adjusted instead.

13. APPROVALS

Role	Name	Signature
Project Supervisor	Md. Anwarul Kabir	
Group Member	MD. AL AMIN	
Group Member	ZANNATUN NUR	
Group Member	MD.REDUONE AHMED	
Group Member	SANMUN NAHAR JOLY	